Authority, Evidence and Durable Continuity: A Five-Cycle Rust Experiment in ESS-MAI

Bledar Gjata
Independent researcher, Gjata Legacy, Albania
ORCID 0009-0003-7854-266X
Research preprint · Capsule version 0.1.0 · 30 October 2026

Abstract

POC011 examines whether a bounded sequence of coordination, candidate elimination and current-evidence verification can preserve missing evidence and counterexamples while admitting eligible actions without duplicating their local recorded effects. The Rust capsule adapts the ESS-MAI method and extracts selected single-use gate code and a later neutral trigger primitive with recorded source hashes. Candidates are integer affine functions over five declared inputs. A five-cycle fixture produces scalar verification results 0,0,0,1,1 and cumulative durable-effect counts 0,0,0,1,2. The recorded validation comprises 38 Rust tests executed in both debug and release and 17 Python verifier self-tests. The experiment materializes limited mechanisms associated with the proposed AI Authority paradigm; it does not establish the full native authority entity, constitutional hierarchy or general scientific validity. Its contribution is a reproducible account of evidence separation, mandatory verification and local continuity, including explicit integrity, rollback and integration limits.

1. Research question and architectural proposal

ESS-MAI proposes an AI Authority as a computational entity organised around a duty, independent executive competence, jurisdiction and governing law. An authority performs work: it may coordinate, reason, measure or verify. Permission or refusal follows from executing that duty; a permission token alone is not the entity. Independence remains bounded by law and jurisdiction. Hierarchical position does not automatically confer every subordinate competence.

The intended organisation comprises GCL, Supreme, laws and hierarchical authorities. GCL denotes the constitutional order, rather than an extra executive voter. The research direction is a governing core or integration component for other systems. POC011 tests only a limited flow, not this complete organisation or an external agent integration. Its origin is the ESS-MAI public project and archived collection, DOI 10.5281/zenodo.22750188; that DOI does not identify POC011.

The question is whether distinct competences can preserve an auditable continuity of evidence while refusing an unsupported action and admitting a supported one. Several calls can perform different competences within a cycle. Coordination, handover and verification are not duplicate execution merely because they inspect related objects. A new cycle may exercise a competence again with current checks. This experiment does not establish a general once-per-cycle authority-entity invariant.

2. Established techniques and declared contribution

Protected subsystems, capabilities, fail-safe defaults and complete mediation have established precedents in computer protection research [1]. SHA-256 is a standard digest algorithm [2], and atomic compare-and-exchange is a documented programming primitive [3]. State machines, journals and deterministic checking are also established techniques; they are not claimed as inventions here.

Gjata's declared contribution is the proposed constitutional organisation of executive competences and the documented assembly tested in this capsule: current-case checking, retained gaps and counterexamples, single-use local admission and durable report/effect continuity. Historical novelty and general architectural correctness remain research questions. No comparative performance claim follows from these tests.

3. Method and extraction provenance

LIGHT binds the current input, context and candidate identities. It retrieves at most eight references from earlier records in that context. QUANTUM evaluates candidates, retains missing samples and contradictions, eliminates inadmissible candidates and selects an eligible proposal. SHADOW recomputes the current assessment and verifies the proposal. Its Spine checks precede Matrix checks; mandatory results are conjoined rather than averaged or voted upon.

D-BAB records actual preconditions before the corresponding operation and the observed outcome afterward. It grants no competence and casts no veto. The 0-NOI primitive supplies a borrowed neutral trigger; the calling operation retains its laws and checks. SHADOW's recomputation exercises verification, not QUANTUM's proposal responsibility.

Selected token, handle and atomic-slot blocks from canonical ESS-MAI Light `sovereign/lgc_gate.rs` are preserved byte-for-byte. The provenance manifest identifies the original SHA-256, inclusive source ranges and extracted-file SHA-256. A wrapper adds bounded issuance, exact binding comparison and process-local gate separation. The small 0-NOI primitive is copied exactly from a later AIGovern-Cyber materialization, not attributed to canonical v1.8.9.

The affine model, stage orchestration, witness integration, transcript verifier, persistence frame and restart lifecycle are explicit adaptations. The capsule does not import the complete native commissioned-authority lifecycle or reproduce every native constitutional boundary. Original source trees were inspected and hashed, not modified. `SOURCE_MAP.md` and the provenance manifests distinguish these categories.

4. Finite model and admission conditions

Let D = {0,1,2,3,4}. Candidate c proposes integer coefficients a and b, with absolute values at most 1,000,000, and samples E_c. Its model and ranking quantities are:

```
f_c(x) = a*x + b
Gain(c) = sum over x in D of f_c(x)
Risk(c) = max over x in D of abs(f_c(x))
```

Evidence must cover every domain point and match the computed values. The current law supplies scope, risk limit, logical payload budget and elapsed-time budget. A candidate is eligible only if its evidence is complete, all samples match and every model value satisfies the risk limit. Eligible candidates are ordered by gain descending, risk ascending and candidate SHA-256 ascending. High gain cannot compensate for a failed condition; the hash is only a deterministic tie-breaker.

An absent law clause yields `HoldIncompleteLaw`. Missing samples yield `HoldMissingEvidence` when no eligible candidate exists and no supplied counterproof determines the outcome. An evidenced limit violation yields `FailClosedCounterproof`. These are capsule status names, not universal native authority semantics. A missing observation is a gap, not a refutation. A limit-violating observation is a bounded counterexample to admissibility, not a disproof of affine mathematics. Samples here are controlled fixtures rather than independent empirical measurements.

The final scalar is 1 only when all mandatory current-case gates and the local single-use boundary succeed. This scalar is not the native ESS-MAI/AIGovern-Cyber paired verdict protocol. Incomplete competitors cannot invalidate an otherwise eligible selected action, but their questions remain recorded.

5. Integrity, memory and local execution

Domain-separated, length-prefixed SHA-256 links context, input, candidate identities, ordered LGC steps and prior receipts. SHADOW checks both these bindings and the recomputed content. The `500` seal binds the current trace and input; an intact refusal can retain an integrity seal without becoming a positive verdict. Neither the seal nor a digest authenticates an issuer.

LIGHT's historical references support retrieval only. Earlier approval cannot satisfy a current evidence requirement. Reopening the store validates journal structure and integrity; this storage task is distinct from SHADOW's current-case adjudication. The capsule performs bounded full replay on reopen and makes no indexing-speed claim.

The selected five values are written as 40 logical bytes into process-owned RAM, read back and hashed. Elapsed time is recorded and checked cooperatively. This is real memory work, but not measurement of physical DRAM latency, energy, temperature or total allocation. The physical-energy field remains absent.

Copied FFI handle bytes refer to one consumable slot. Exact binding and nonce checks precede atomic consumption; at most one successful consumption is possible for that slot. The wrapper permits at most 1,024 lifetime actions per gate and keeps the native token private. This protects a particular admission, not the definition or transfer of executive competence. The C-ABI probe is internal Rust code using the C calling convention, not a separately compiled external C client.

Each framed journal record contains the report and optional local effect. After write, flush, synchronization and identical readback, memory state becomes visible and transient in-flight state is released. Output and questions then return for delivery. A delivery failure cannot undo a durable commit. The effect key combines context and action identity: identical repetition adds no effect, while conflicting values refuse commit. This describes local recorded effects, not distributed exactly-once execution.

6. Executed results

Validation recorded on 30 October 2026 used Windows 10 AMD64 and Cargo 1.98.0. Formatting and release build succeeded. All 38 Rust tests passed in debug and again in release: 33 library, three CLI and two process integration tests. These are two executions of the same tests, not 76 distinct properties. A separately implemented Python verifier passed with 17 self-tests; this is software cross-checking, not independent peer review.

Cycle	Current material	Scalar	Cumulative effects
1	Risk-limit clause absent	0	0
2	Limit 10; required samples absent	0	0
3	Complete samples for 4x+1; risk 17	0	0
4	Complete samples for 2x+1; gain 25, risk 9	1	1
5	New action 2x+2; gain 30, risk 10	1	2

Cycles 4 and 5 retain the inadmissible competitor and its two counterexamples. Success neither erases prior evidence nor approves the next cycle. Adversarial checks reject altered assessments, reordered or rehashed role traces, corrupted hardware receipts and invalid journal material. Eight copied handles admit one consumption; 32 concurrent attempts produce one winner. Wrong bindings and nonces do not consume the legitimate slot. Separate CLI-process cycles reconstruct the context without duplicating an identical local effect. `RESULTS.md` links exact commands, logs, fixture outputs and development failures corrected before the final run.

7. Reproduction and limits

From the capsule root, run:

```
cargo test --locked --offline
cargo test --locked --offline --release
cargo run --locked --offline --release -- demo demo-run
python tools/verify.py demo-run --self-test
```

The demo directory must not already exist. Offline execution requires the pinned dependencies in the local cache. Mathematical selection is deterministic; timings and dependent receipt hashes can differ across executions. Recorded hashes identify recorded artifacts, not every future run.

The trusted boundary includes the program, host and local storage. Replacing the complete journal with an older valid prefix is undetectable without an external trusted head. Synchronization and readback do not establish physical power-loss resilience. No hostile-host resistance, external service effect, complete KiS/KiC pipeline, native authority-entity proof or VERITAS integration is demonstrated.

AI-assisted implementation was conducted under the author's architecture and direction, with human source review and recorded reproducible checks. Those checks are not independent scientific validation. Included code and documentation are offered under Apache-2.0; excluded private runtime material is not relicensed. The publication repository is ESS-MAI-POC011-Authority-Evidence-Continuity. The next evaluation should specify an external interface, independently observed effects and adversarial scenarios before making broader integration claims.

References

1. J. H. Saltzer and M. D. Schroeder. *The Protection of Information in Computer Systems*. 1975. Author-hosted text.

2. National Institute of Standards and Technology. *Secure Hash Standard (SHS)*. FIPS 180-4, August 2015. Official publication. DOI: 10.6028/NIST.FIPS.180-4.

3. The Rust Project Developers. *Standard library atomic types: compare_exchange*. Official documentation. Accessed 30 October 2026.